

THUMOS14 Temporal Action Detection: A Two-Stage Pipeline with I3D and VideoMAE v2 Backbones

Anthony Stefanuto

*Department of Computer
Science*

Western University
London, ON, Canada
astefanu@uwo.ca

Alex Caraman

*Department of Computer
Science*

Western University
London, ON, Canada
acaraman@uwo.ca

Jerry Kang

*Department of Computer
Science*

Western University
London, ON, Canada
jkang365@uwo.ca

Guojia La

*Department of Computer
Science*

Western University
London, ON, Canada
gla2@uwo.ca

Code Repository:

<https://github.com/Guojia2/CS4452>

Abstract—Temporal action detection (TAD) requires localizing and classifying every action instance in untrimmed videos. Recent advances along two independent axes, video backbone representations and temporal detection heads, each report substantial gains, creating the expectation that combining a stronger backbone with a stronger detector should yield compounding improvements. This work tests that assumption through a controlled 2×2 factorial study on the THUMOS14 benchmark. We cross two pre-extracted feature backbones, I3D (2048-dimensional, supervised 3D CNN) and VideoMAE v2 (768-dimensional, self-supervised masked autoencoder), with two temporal detection heads, a purely convolutional CNN baseline and an ActionFormer-style conv-transformer detector with local windowed self-attention. All four configurations share the same localization framework, training objective, and evaluation protocol; only the detection head is trained. Our results reveal that ActionFormer achieves 66.76% average mAP with I3D features but collapses to 4.57% with VideoMAE v2, a near-total failure rather than a gradual degradation. Conversely, the CNN baseline performs better with VideoMAE v2 (4.04%) than with I3D (1.66%), though this comparison is confounded by a stride difference (stride 8 vs. stride 16) that gives VideoMAE v2 twice the temporal resolution. We identify feature stride and pyramid-depth differences as confounding variables that co-vary with backbone and head choices across conditions, limiting the causal conclusions that can be drawn. Nevertheless, the results suggest that temporal detection models and feature backbones are tightly coupled, and that advances in representation learning do not translate directly into localization gains without architectural or training adaptation.

I. INTRODUCTION

Temporal action detection (TAD) is the task of identifying every action instance in an untrimmed video by predicting its start time, end time, and semantic category. Unlike action classification on pre-trimmed clips, TAD must jointly solve localization and recognition over long video streams in which actions occupy only a small fraction of the total duration. This makes TAD both practically important and technically

demanding, with applications in sports analytics, surveillance, and video retrieval.

Recent progress in TAD has advanced along two largely independent research axes. On the representation side, video backbones have evolved from two-stream 2D CNNs [7] and 3D convolutional networks such as C3D [8] and I3D [9] to self-supervised vision transformers like VideoMAE [13], which learn strong representations by reconstructing heavily masked video patches without labeled data. On the temporal modeling side, detection heads have progressed from two-stage proposal-then-classify pipelines [2]–[4] to single-stage anchor-free detectors such as ActionFormer [5], which applies multi-scale self-attention over temporal feature pyramids, and TemporalMaxer [6], which achieves comparable accuracy using only parameter-free max-pooling. Each line of work reports substantial gains on standard benchmarks, and the natural expectation is that combining a stronger backbone with a stronger detector should yield compounding improvements.

However, this expectation is rarely tested directly. Most published TAD systems change both the backbone and the detection head simultaneously, making it difficult to attribute performance gains to one component or the other. Furthermore, modern detectors are typically developed and tuned against a single feature type, commonly I3D features pre-trained on Kinetics [9], raising the question of whether their advantages generalize across backbones or reflect implicit co-optimization with a specific feature distribution.

This work investigates that question on THUMOS14 [1], the standard TAD benchmark comprising 413 untrimmed sports videos annotated across 20 action classes. We construct a 2×2 comparison crossing two pre-extracted feature backbones, I3D, a classical 3D CNN producing 2048-dimensional clip features, and VideoMAE v2, a modern masked autoencoder producing 768-dimensional features, with two temporal detection heads. In short, we are testing a purely convolutional CNN baseline and an ActionFormer-style conv-transformer detector with local windowed self-attention. All four configurations share the same dense point-based localization framework, focal loss

[16] classification objective, DIOU regression loss, AdamW optimizer, and evaluation protocol. Features are pre-extracted and frozen so that only the lightweight detection head is trained, following the decoupled pipeline design advocated by OpenTAD [17].

Our results reveal a striking asymmetry. ActionFormer paired with I3D features achieves 66.76% average mAP, consistent with the published ActionFormer result on THUMOS14. Replacing I3D with VideoMAE v2 features causes performance to collapse to 4.57%. This is a near-total failure across all tIoU thresholds rather than a gradual degradation. Meanwhile, the simpler CNN baseline performs better with VideoMAE v2 features (4.04% mAP) than with I3D (1.66% mAP), though this comparison is confounded by a stride difference (stride 8 for VideoMAE v2 vs. stride 16 for I3D) that gives VideoMAE v2 twice the temporal resolution. More broadly, feature stride, the number of feature-pyramid levels, and detection head capacity all vary across conditions and co-vary with the backbone and head choices, limiting the causal conclusions that can be drawn from any single pairwise comparison. Nevertheless, the overall pattern of results suggests that temporal detection models and feature backbones are tightly coupled, and that advances in representation learning do not translate directly into localization gains without architectural or training adaptation.

The remainder of this paper is organized as follows. Section II surveys related work in temporal action detection, video backbone architectures, temporal modeling heads, and decoupled feature extraction pipelines. Section III describes our experimental methodology. Section IV presents and analyzes results. Section V discusses implications and limitations, and Section VI concludes.

II. BACKGROUND & RELATED WORK

A. Temporal Action Detection

Temporal action detection (TAD) requires predicting the start time, end time, and category of every action instance in an untrimmed video. THUMOS-14 [1] established the standard evaluation protocol using mean Average Precision (mAP) at temporal Intersection over Union (tIoU) thresholds of 0.3–0.7 across 20 action classes in 413 untrimmed sports videos, the primary benchmark for this work.

Early TAD research found an approach of a two-stage paradigm consisting of proposal generation followed by classification. Deep Action Proposals (DAPs) [2] used deep recurrent networks over pre-extracted features for proposal generation, showing that temporal detection can be built on fixed video features. Boundary-based methods improved proposal generation. Boundary Sensitive Network (BSN) [3] used one-dimensional (1D) temporal convolutions to improve boundary precision, while Boundary-Matching Network (BMN) [4] utilized a 2D confidence map over all start-end pairs to assess proposal quality over all temporal boundaries. However, both methods still rely on fixed anchor scales and separate classification stages.

Anchor-free transformer detectors have since become the preferred approach. ActionFormer [5], which serves as the transformer head in our pipeline, combines a multi-scale temporal feature pyramid with self-attention for single-shot detection and improves THUMOS-14 mAP@0.5 by 14.1 points over previous work. TemporalMaxer [6] replaces self-attention with parameter-free max-pooling, matching ActionFormer’s performance while requiring $2.8\times$ fewer giga multiply-accumulate operations (GMACs) and achieving $3\times$ faster inference, indicating that strong backbone features may reduce the need for complex temporal modeling. Our CNN baseline head uses a 1D convolutional architecture that operates over sliding windows of clip-level features, providing a lightweight comparison point against ActionFormer.

B. Video Backbone Architectures

Backbone feature quality plays a major role in TAD performance. Simonyan and Zisserman [7] introduced a two-stream approach, which combines spatial RGB information with temporal motion from optical flow. Tran *et al.* [8] combined spatial and temporal modeling in C3D, a 3D CNN creating a segment-based descriptor framework that remains used in modern TAD pipelines. Carreira and Zisserman [9] extended this idea by converting pretrained 2D image filters into 3D video filters, creating Inflated 3D ConvNets (I3D). They also introduced Kinetics, a large-scale action recognition benchmark used to pretrain both backbones in this work. I3D has since become the de facto standard feature extractor for TAD on THUMOS-14: the OpenTAD benchmark [17] uses two-stream I3D features as its default for all THUMOS-14 experiments, and the public configurations of ActionFormer [5], TriDet [19], and TemporalMaxer [6] are all tuned and validated on I3D features. We use I3D as our 3D CNN backbone precisely because of this ecosystem-wide adoption, which allows us to examine how tightly modern detection heads depend on a single feature distribution. SlowFast Networks [10] further proved that temporal information can be utilized effectively by a fast pathway using only $\sim 20\%$ of the total computation. More recently, X3D [11] demonstrated that efficient 3D CNN architectures can be systematically expanded along multiple axes to balance accuracy and cost.

For self-supervised video transformers, He *et al.* [12] showed that masking 75% of image patches and then reconstructing them produces strong representations without labeled data. Tong *et al.* [13] leveraged this finding to videos in Video Masked Autoencoders (VideoMAE), increasing the masking ratio to 90–95% by exploiting the redundancy between neighboring video frames. Wang *et al.* [14] scaled this approach in VideoMAE v2 by introducing a dual masking strategy that masks both the encoder and decoder, reducing memory and computation enough to train billion-parameter video transformers. VideoMAE v2 also adopts a progressive training paradigm: initial pretraining on a large, diverse unlabeled dataset followed by post-pretraining on a labeled mixture, achieving state-of-the-art results on Kinetics-400 (90.0%) and Something-Something V2 (77.0%). We use VideoMAE v2

as our vision transformer (ViT) backbone so that we can directly compare self-supervised transformer features with the supervised 3D CNN features from I3D.

C. Temporal Modeling Heads and Training Objectives

Abu Farha and Gall [15] introduced the Multi-Stage Temporal Convolutional Network (MS-TCN), an architecture based on dilated 1D temporal convolutions that captures long range temporal context without temporal downsampling. Dilated 1D temporal convolutions are time-based filters that skip over some steps so they can capture patterns across a longer stretch of video. Its multi-stage design influenced the pyramid architectures used by both ActionFormer and TemporalMaxer. To address the class imbalance between background and action clips, Lin *et al.* [16] introduced Focal Loss, which dynamically lowers the weights of easy negatives through the factor $(1 - p)^\gamma$. We apply Focal Loss with $\alpha = 0.25$ and $\gamma = 2.0$ to multi-hot clip labels, treating each class as a separate yes/no prediction so that multiple actions can be assigned to the same clip.

D. Decoupled Feature Extraction Pipelines

Separating backbone feature extraction from temporal head training makes experiments faster. Video features are extracted once with a frozen backbone, and only the lightweight head is trained afterward. Liu *et al.* [17] utilized this strategy in OpenTAD, a framework that covers 16 TAD methods, including ActionFormer and TemporalMaxer, and 9 datasets, including THUMOS-14, while using pre-extracted features from various backbones as a benchmark standard. We follow the same design: features are stored on a Modal cloud volume and reused across all experiments, reducing training time significantly per run on a single NVIDIA A10G GPU.

E. Research Gap

Most published TAD systems are massive end-to-end pipelines that require full retraining for each experiment, making model comparison studies computationally expensive. It is often unclear whether performance differences come from the backbone or the head, since both usually change across papers. We address this limitation by holding backbone features constant when comparing heads (ActionFormer vs. CNN baseline) and holding the head constant when comparing backbones (I3D vs. VideoMAE v2). This design allows us to examine how performance changes with each component, while acknowledging confounds such as stride and pyramid depth.

III. METHODS

A. Research Objectives

This study pursues three objectives:

- O1. Quantify backbone-detector interaction.** Measure how swapping the feature backbone (I3D vs. VideoMAE v2) affects detection performance for each temporal head, and vice versa, using a controlled factorial design on THUMOS14. This is significant because most

published comparisons change both components simultaneously, obscuring the source of performance gains.

- O2. Test the composability of independent advances.**

Determine whether pairing a modern backbone (VideoMAE v2) with a modern detector (ActionFormer) yields additive improvements or whether the two components require co-adaptation. This has practical implications for researchers who wish to upgrade individual pipeline stages without full system redesign.

- O3. Identify confounding variables.** Document which configuration differences beyond the backbone and head, such as feature stride, pyramid depth, and head capacity, vary across conditions and may affect the attribution of performance differences. This informs the design of cleaner future comparisons.

B. Study Design

This work evaluates temporal action detection through a structured 2×2 comparison across two independent axes: the choice of feature backbone and the choice of temporal detection model. For the backbone, we compare a classical 3D convolutional network (I3D) against a modern transformer-based video representation model (VideoMAE v2). For the temporal detector, we compare a convolutional baseline against an ActionFormer-style point-transformer detector. This factorial design allows us to isolate the contribution of each component and assess whether improvements in feature learning and temporal localization compose additively.

C. Input Representation

Both detector architectures operate on pre-extracted sequences of clip-level feature vectors rather than raw video frames. This two-stage approach is standard in the temporal action detection literature and decouples representation learning from temporal localization, enabling controlled comparison across backbone and detector choices.

In the I3D setting, each clip is encoded as a 2048-dimensional feature vector using a two-stream Inflated 3D ConvNet pre-trained on Kinetics-400 [9]. In the VideoMAE v2 setting, clip features are 768-dimensional vectors produced by a VideoMAE v2 Base masked autoencoder pre-trained on large-scale video data [14]. In both cases, each feature vector corresponds to a span of 16 frames. The temporal density of the resulting feature sequence is governed by a configurable stride parameter, which differs across experimental conditions (see Table I). All localization is performed directly in the temporal feature grid: ground-truth action intervals are converted from video time into feature-grid coordinates, predictions are made in this same space, and predicted boundaries are mapped back to video time during inference.

D. Temporal Detection Models

1) CNN Baseline: The classical baseline uses a purely convolutional temporal backbone with architecture parameters [1, 1, 3] (1 embedding block, 1 stem block, 3 branch blocks). The architecture consists of masked 1D convolutional

embedding layers at the input stage, followed by a series of residual temporal convolution blocks. Progressive temporal downsampling forms a multi-scale feature hierarchy with 4 pyramid levels, which is consumed by convolutional classification and regression heads for dense prediction. As detailed in Section IV-A2c, each head consists of one intermediate convolutional layer followed by a final output layer. This model relies entirely on local temporal aggregation through convolutions and uses no positional encoding or self-attention.

2) *ActionFormer*: The modern detector follows an ActionFormer-style conv-transformer architecture with architecture parameters [2, 2, 5] (2 embedding blocks, 2 stem blocks, 5 branch blocks). The input stage uses convolutional embedding to project clip features into a 512-dimensional hidden space. This is followed by transformer blocks for temporal modeling, which use local multi-head self-attention with a window size of 19 (i.e. ± 9 neighbors) rather than full global attention, improving computational efficiency while preserving the ability to model non-local temporal context. Multi-scale temporal features are generated through downsampling transformer stages, forming a 6-level feature pyramid. Classification and regression heads share the same architecture described in Section IV-A2c and are applied densely at each temporal location across all scales.

E. Experimental Configurations

Table I summarizes the four experimental conditions. All configurations share the same embedding dimension (512), optimizer, loss functions, and evaluation protocol. The primary differences are input dimensionality, feature stride, backbone architecture depth, the number of feature pyramid levels with their associated regression ranges, and regularization: ActionFormer applies drop-path regularization with rate 0.1 in its transformer blocks, while the CNN baseline uses no drop-path.

TABLE I

SUMMARY OF THE FOUR EXPERIMENTAL CONFIGURATIONS. ALL OTHER HYPERPARAMETERS (OPTIMIZER, LEARNING RATE, EPOCHS, BATCH SIZE) ARE IDENTICAL ACROSS CONDITIONS.

Config	Backbone	Head	Dim	Stride	Levels
AF+I3D	I3D	ActionFormer	2048	4	6
AF+VMv2	VideoMAE v2	ActionFormer	768	8	6
BL+I3D	I3D	CNN Baseline	2048	16	4
BL+VMv2	VideoMAE v2	CNN Baseline	768	8	4

We note that the feature stride is not held constant across all conditions: I3D features are extracted at stride 4 for ActionFormer and stride 16 for the CNN baseline, while VideoMAE v2 features use stride 8 in both cases. Similarly, ActionFormer operates with a 6-level pyramid while the baseline uses 4 levels. These differences reflect the configurations under which each detector was developed and are preserved to evaluate each model in its intended operating regime. However, they represent confounds that should be considered when interpreting performance differences across conditions. In particular, the stride difference between BL+I3D (stride 16) and BL+VMv2 (stride 8) means that VideoMAE v2 features

have twice the temporal resolution in the baseline comparison, and the stride difference between AF+I3D (stride 4) and AF+VMv2 (stride 8) means that ActionFormer receives coarser input when operating on VideoMAE v2 features. These stride gaps may account for some or all of the observed performance differences in those pairings.

When switching between backbones, no explicit adaptation or projection layer is added. The first embedding convolution (a 1D convolution with kernel size 3 and padding 1) directly maps the input dimensionality, 2048 for I3D or 768 for VideoMAE v2, into the shared 512-dimensional embedding space. Both architectures apply LayerNorm after embedding, within the feature pyramid, and in the detection heads.

F. Point-Based Localization Framework

Both detectors operate within a shared dense point-based localization framework. At each temporal location and scale, the model jointly predicts an action-category confidence score and boundary offsets relative to that point. Supervision is assigned according to whether a temporal point falls inside a ground-truth action interval and within the valid regression range for its temporal scale. This formulation enables localization to be learned jointly across multiple temporal resolutions in a fully convolutional and transformer-compatible manner, without requiring separate proposal generation or two-stage processing.

G. Training Configuration

All four configurations are trained with identical optimization settings. We use AdamW [18] with a learning rate of 1×10^{-4} and weight decay of 0.05. A cosine annealing schedule is applied over the remaining 30 epochs after a 5-epoch linear warmup, for a total of 35 training epochs. The batch size is 2. An exponential moving average (EMA) of model weights is maintained with a decay factor of 0.999 and used for evaluation.

Classification is supervised using sigmoid focal loss [16] with $\alpha = 0.25$ and $\gamma = 2.0$, treating each of the 20 action classes as an independent binary prediction so that overlapping actions can be assigned to the same temporal location. Localization is supervised using a distance-IoU (DIoU) regression loss applied to predicted boundary offsets, computed only over positive (foreground) locations. The regression loss is normalized by an exponential moving average of the number of positive samples per batch, initialized at 100 with a momentum of 0.9. During training, temporal feature sequences are randomly cropped to 90–100% of their length as a data augmentation step; this is applied identically across all four conditions. A `label_smoothing` parameter is present in the configuration defaults but is not connected to the training loop; focal loss operates on raw multi-hot labels in all conditions.

Training is performed on a single NVIDIA A10G GPU via the Modal cloud computing platform. Features are pre-extracted once per backbone and stored on a Modal cloud

volume, so that only the lightweight detection head is trained in each experiment.

H. Post-Processing

During inference, predicted segments are refined using soft non-maximum suppression (Soft-NMS) with Gaussian decay, an IoU threshold of 0.1, and $\sigma = 0.5$. Up to 2000 candidate segments per video are retained before NMS, with a minimum pre-NMS score threshold of 0.001. After suppression, a maximum of 200 segments per video are kept, with a minimum score of 0.001 and a minimum duration threshold of 0.05. NMS is applied independently per action class (multiclass NMS). Segment voting is available in the codebase but is disabled when multiclass NMS is active, as per-class candidate counts are insufficient for reliable boundary refinement; it is not applied in any of the four experimental conditions.

I. Evaluation Protocol

All models are evaluated using temporal intersection over union (tIoU) between predicted and ground-truth segments. A predicted segment is considered a true positive if its tIoU with a ground-truth segment of the same action class meets or exceeds a specified threshold. Performance is reported using mean Average Precision (mAP) computed across the 20 THUMOS14 action classes at five tIoU thresholds: $\{0.3, 0.4, 0.5, 0.6, 0.7\}$. The training split consists of the 200 THUMOS14 validation videos (3003 annotated segments), and the test split consists of 211 annotated test videos. This evaluation protocol follows the standard THUMOS14 benchmark convention used in prior work [1].

IV. RESULTS

A. System Architecture and Design

Our pipeline follows a decoupled two-stage design: features are extracted once per backbone and stored as per-video `.npy` files on a Modal cloud volume, then a lightweight detection model is trained on these frozen features. This separation means that switching backbones or heads requires no re-extraction, and each training run completes in under 12 hours on a single GPU.

1) *Feature Extraction (Stage 1)*: Both backbones consume 16-frame clips but differ in temporal stride, feature dimensionality, and extraction origin.

- **I3D**. An Inception-v1 architecture inflated to 3D and pretrained on Kinetics-400, producing 2048-dimensional features per clip. These features were pre-extracted externally and are the same files used by the official ActionFormer release. We use a temporal stride of 4 (75% overlap) for the ActionFormer head and 16 (no overlap) for the CNN baseline, matching their respective published configurations.
- **VideoMAE v2**. A ViT-Base model (OpenGVLab/VideoMAEv2-Base) loaded via Hugging Face Transformers. Each 16-frame clip is preprocessed with the model’s native image processor, passed through the frozen encoder, and

the `last_hidden_state` is mean-pooled over all spatial-temporal tokens to yield a 768-dimensional feature vector. A temporal stride of 8 (50% overlap) is used for both heads. If a clip has fewer than 16 frames, the last frame is repeated to pad the input.

At loading time, no additional normalization, temporal interpolation, or zero-padding is applied; raw `float32` arrays are transposed from $T \times C$ to $C \times T$ and passed directly into the model. Long sequences are cropped to a maximum of 2304 timesteps during training.

2) *Detection Model (Stage 2)*: All four configurations share the same overall architecture (PTTRANSFORMER), which consists of three components: an encoder backbone, a feature pyramid neck, and twin detection heads.

a) *Encoder backbones*: The difference between our ActionFormer and CNN baseline configurations lies entirely in the encoder, not in the detection heads.

- **ConvTransformerBackbone (ActionFormer)** uses a layer configuration of (2, 2, 5): two embedding convolutions, two stem blocks, and five transformer branch blocks with local self-attention (window size 19), producing a 6-level feature pyramid.
- **ConvBackbone (CNN baseline)** uses a configuration of (1, 1, 3): one embedding convolution (`MaskedConv1D`, kernel size 3, stride 1, followed by `LayerNorm` and `ReLU`), one stem `CONVBLOCK` at stride 1, and three branch `CONVBLOCKS` that each downsample by a factor of 2 via strided convolution, producing a 4-level feature pyramid. Each `CONVBLOCK` is a ResNet-style residual unit with two 1D convolutions (kernel 3, expansion factor 2 giving an internal width of 1024), a residual connection (with a 1×1 strided projection when downsampling), and `ReLU` activation.

Both backbones project input features to a shared embedding dimension of 512 via the first convolutional layer, so the downstream neck and heads are agnostic to the original feature dimensionality (2048 for I3D, 768 for VideoMAE v2).

b) *Neck*.: All configurations use an identity FPN (`FPNIDENTITY`) that applies only `LayerNorm` at each pyramid level, with no lateral connections or top-down pathway.

c) *Detection heads*.: Classification and regression share the same tower architecture across both the ActionFormer and CNN baseline configurations: one intermediate `MaskedConv1D` layer (kernel 3, 512 channels, `LayerNorm`, `ReLU`) followed by a final `MaskedConv1D` output layer. The classification head outputs 20 channels (one per THUMOS class) with sigmoid activation for multi-label prediction. The regression head outputs 2 channels (left and right temporal offsets), scaled by a learnable per-level `SCALE` parameter and passed through `ReLU` to enforce non-negative offsets. The baseline uses 4 FPN levels with regression ranges $[0, 4]$, $[4, 8]$, $[8, 16]$, $[16, 10000]$, while ActionFormer uses 6 levels with ranges extending to $[64, 10000]$.

d) *Post-processing*.: Predicted offsets are decoded into temporal segments in seconds, filtered by a pre-NMS score

threshold of 0.001 and a top- k limit of 2000 per FPN level. Per-class soft NMS is applied with an IoU threshold of 0.1 and Gaussian sigma of 0.5, retaining at most 200 segments per video.

B. Implementation and Testing

1) *Codebase and Framework*: The ActionFormer implementation is vendored from the official HKU-MMLab repository into a local `libs/` directory and adapted for our experiments. The core detection architecture (the PTTRANSFORMER, detection heads, focal and DIoU losses, and the soft NMS pipeline) is structurally unchanged from the original release. We added the CONVBACBONE as a conv-only encoder variant, a VideoMAE v2 feature extraction module using Hugging Face Transformers (v4.30.2), and Modal cloud infrastructure wrappers for GPU-based training, evaluation, and extraction. A custom C++ NMS extension (`libs/utils/csrc/nms_cpu.cpp`) is compiled at image build time for efficient CPU-based NMS. The full stack runs on PyTorch with a single NVIDIA A10G GPU provisioned through Modal.

2) *Training Configuration*: All four configurations share the same optimizer and schedule: AdamW with a learning rate of 10^{-4} , weight decay of 0.05 (applied selectively to convolution and linear weights, excluding biases, LayerNorm parameters, and SCALE parameters via minGPT-style parameter grouping), 5 warmup epochs with linear ramp, and 30 post-warmup epochs with cosine annealing, for a total of 35 training epochs. The batch size is 2. An exponential moving average (EMA) of model weights is maintained with a decay of 0.999, and the EMA checkpoint is used for evaluation. The CNN baseline applies L2 gradient clipping at norm 1.0, while ActionFormer disables gradient clipping, relying on pre-LayerNorm transformer stability. Classification is supervised with sigmoid focal loss ($\alpha = 0.25$, $\gamma = 2.0$) over multi-hot clip labels, and regression uses a 1D Distance-IoU loss with equal weight. A center sampling radius of 1.5 assigns positive training targets. All configurations use the same random seed (1234567891).

3) *Validation of Setup*: The I3D + ActionFormer configuration is designed to closely reproduce the published ActionFormer result on THUMOS14. It uses the same feature stride (4), the same backbone architecture (2, 2, 5 with window size 19), the same identity FPN, the same loss functions, the same soft NMS pipeline, and the same annotation format (`thumos14_af.json`) and evaluation protocol. Our AF+I3D configuration achieves 66.76% average mAP, which is consistent with the published result of approximately 66.8%, providing confidence that this path of the pipeline functions as intended.

4) *Evaluation Protocol*: Evaluation follows the standard THUMOS-14 protocol using the ANETDETECTION evaluator adapted from the official ActivityNet evaluation code. The model produces predictions in ActivityNet format (video-id, start, end, label, score), and interpolated average precision (Pascal VOC style) is computed per class at tIoU thresholds of

TABLE II
MAP (%) ON THUMOS-14 AT FIVE tIoU THRESHOLDS. BEST RESULTS
PER COLUMN ARE IN **BOLD**.

Head	Backbone	0.3	0.4	0.5	0.6	0.7	Avg.
ActionFormer	I3D	82.09	78.35	71.25	58.27	43.83	66.76
ActionFormer	VideoMAE v2	8.86	6.69	4.31	2.20	0.80	4.57
Baseline	VideoMAE v2	7.96	5.79	3.76	1.83	0.85	4.04
Baseline	I3D	3.54	2.36	1.41	0.72	0.27	1.66

{0.3, 0.4, 0.5, 0.6, 0.7}. Per-class APs are averaged to produce mAP at each threshold, plus an overall average mAP.

5) *Known Issues*: During development we identified two implementation issues worth noting.¹ The THUMOS-14 annotation JSON uses inverted subset names (“training” maps to the validation folder, “validation” to the test folder), which the YAML configs handle correctly but which is a common source of confusion when working with this dataset.

C. Evaluation Results

We evaluate four pipeline configurations on the THUMOS-14 test set by combining two backbones (I3D, VideoMAE v2) with two temporal heads (ActionFormer, CNN baseline). All configurations share the same pre-extracted features per backbone, isolating the effect of each component. Table II reports mAP and Table III reports Recall at tIoU thresholds 0.3–0.7, following the standard THUMOS-14 evaluation protocol.

D. ActionFormer + I3D Dominates

ActionFormer paired with I3D features achieves an average mAP of 66.76%, outperforming every other configuration by a wide margin (Fig. 1). Its advantage holds across all tIoU thresholds, from 82.09% at 0.3 down to 43.83% at 0.7. Recall follows the same pattern (Table III): Recall@1x reaches 84.35% at tIoU 0.3 and remains above 53% even at the strictest threshold, indicating that the model localizes most action instances with tight temporal boundaries. The remaining three configurations all fall below 9% average mAP, a gap unlikely to be explained by minor tuning alone, though targeted tuning was not performed here.

E. Backbone–Head Coupling

The central finding of this work is how strongly ActionFormer’s performance depends on its backbone. Replacing I3D with VideoMAE v2 features drops average mAP from 66.76% to 4.57%, consistently across every threshold (Table II). On I3D features, ActionFormer improves over the CNN baseline by roughly $40\times$ ($1.66\% \rightarrow 66.76\%$), yet on VideoMAE v2 the same swap yields only $1.1\times$ ($4.04\% \rightarrow 4.57\%$), as shown in Fig. 2. However, these two comparisons involve different strides (stride 4 for AF+I3D vs. stride 16 for BL+I3D, and

¹The VideoMAE v2 extraction function defaults to a clip stride of 4 in its function signature, but the Modal endpoint overrides this to 8. Calling the function directly without specifying the stride produces features that silently mismatch the YAML configuration, causing incorrect temporal alignment with no error. Additionally, a latent bug exists in the non-warmup multistep scheduler path that references an undefined variable, though this code path is never triggered since all configurations use warmup.

TABLE III

RECALL (%) ON THUMOS-14 AT RECALL@1X AND RECALL@5X FOR EACH TIOU THRESHOLD. AF = ACTIONFORMER, BL = CNN BASELINE, VM2 = VIDEOMAE v2. BEST RESULTS PER COLUMN ARE IN **BOLD**.

Head	Backbone	0.3		0.4		0.5		0.6		0.7	
		@1x	@5x	@1x	@5x	@1x	@5x	@1x	@5x	@1x	@5x
AF	I3D	84.35	96.62	80.43	95.15	74.82	92.05	64.31	83.62	53.16	70.62
AF	VM2	21.37	29.56	17.97	27.12	14.15	23.44	9.16	18.11	5.01	11.55
BL	VM2	20.63	28.99	17.55	26.54	13.56	22.73	8.89	17.50	5.80	11.45
BL	I3D	9.57	30.70	7.06	23.18	4.73	15.98	2.66	9.73	1.32	5.20

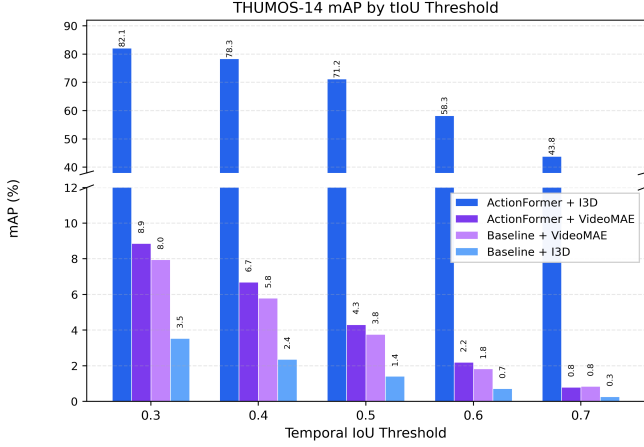


Fig. 1. mAP at each tIoU threshold for all four backbone-head configurations. ActionFormer paired with I3D features substantially outperforms all other combinations at every threshold.

stride 8 for both VideoMAE v2 conditions), so the $40\times$ figure reflects the combined effect of the head, the stride, and the pyramid depth, not the head alone. Nevertheless, the magnitude of the gap suggests that ActionFormer’s architecture and regression ranges are implicitly calibrated to I3D’s feature distribution, and that the head struggles when those assumptions are violated.

F. Backbone Comparison Under the CNN Baseline

Under the CNN baseline, VideoMAE v2 features (4.04% avg mAP) outperform I3D (1.66%) by roughly $2.4\times$, consistently across all five thresholds. However, this comparison is confounded by the stride difference: BL+VMv2 uses stride 8 while BL+I3D uses stride 16, giving the VideoMAE v2 condition twice the temporal resolution. It is therefore unclear whether the performance gap reflects stronger features, finer temporal granularity, or a combination of both. A fair comparison would require running both backbones at matched strides, which we leave to future work.

G. Non-Composability of Improvements

One would expect that pairing a stronger backbone (VideoMAE v2) with a stronger head (ActionFormer) should give the best results. Instead, ActionFormer + VideoMAE v2 (4.57%) barely beats the CNN baseline + VideoMAE v2 (4.04%) and falls far short of ActionFormer + I3D (66.76%). In

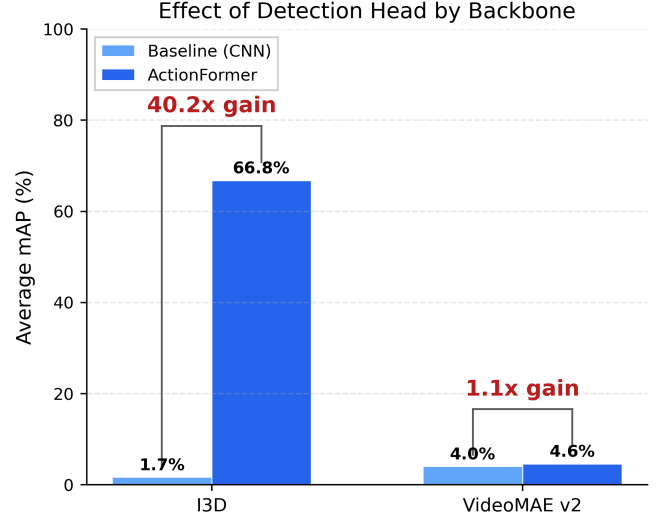


Fig. 2. Effect of switching from the CNN baseline to ActionFormer on average mAP, grouped by backbone. The $\sim 40\times$ gain on I3D features reflects the combined effect of the head, stride, and pyramid depth differences between the two configurations (see Table I). On VideoMAE v2, where both heads share the same stride, the gain is only $\sim 1\times$.

our setup, improvements in backbone and detector design do not compose automatically, because each component was tuned under different assumptions about input distribution and temporal resolution.

H. Limitations

Feature extraction hyperparameters (clip length, temporal stride, spatial resolution) were not independently tuned per backbone; the temporal stride used for I3D ActionFormer (4) differs from VideoMAE v2 (8), and the I3D baseline uses stride 16, which may partially inflate the observed performance gap. The number of pyramid levels also differs between the ActionFormer (6) and baseline (4) configurations, introducing an additional confound. A fairer comparison would involve backbone-specific extraction settings and matched pyramid depths optimized independently, which we leave to future work. Additionally, all experiments use a single random seed, so variance estimates across runs are not available. Because all configurations were run with a single seed, we do not estimate variance across runs, so the conclusions should be interpreted as directional rather than definitive.

V. DISCUSSION

The central finding of this work is that temporal action detection performance is not determined by the backbone or the detection head in isolation, but by their interaction. ActionFormer achieves 66.76% average mAP with I3D features, a strong result on THUMOS14, yet collapses to 4.57% when the same architecture receives VideoMAE v2 features. This is not a modest degradation that might be recovered with minor tuning; it is a near-complete failure of the detection pipeline. Understanding why this occurs, and what it implies for TAD research, is the focus of this section.

A. Sources of Backbone–Detector Coupling

Several properties of the I3D and VideoMAE v2 feature spaces differ in ways that are likely to affect downstream detection.

First, the two backbones produce features with different dimensionalities (2048 vs. 768) and potentially different value distributions. Although the embedding convolution projects both into a shared 512-dimensional space, a single linear projection cannot compensate for differences in feature variance, sparsity, or semantic structure between a supervised 3D CNN and a self-supervised masked autoencoder. ActionFormer’s regression ranges, learning rate, and loss normalization were all calibrated against I3D feature statistics; none of these were adjusted for VideoMAE v2.

Second, the temporal characteristics of the two feature types differ. I3D uses a two-stream architecture that explicitly encodes optical flow, producing features with strong motion-based temporal signals. VideoMAE v2 learns temporal representations implicitly through masked reconstruction, which may encode temporal information in a less directly accessible form for boundary-sensitive regression heads.

Third, the feature stride differs across conditions (stride 4 for ActionFormer+I3D vs. stride 8 for ActionFormer+VideoMAE v2), meaning that the temporal granularity of the input sequence changes with the backbone. This stride difference is a meaningful confound: coarser input resolution directly limits the precision with which boundaries can be predicted, and some portion of the performance gap between AF+I3D and AF+VMv2 may be attributable to stride rather than feature quality.

B. Why Improvements Do Not Compose

A natural assumption in modular system design is that upgrading one component should not degrade overall performance. Our results violate this assumption: combining a modern backbone (VideoMAE v2) with a modern detector (ActionFormer) produces worse results than pairing either component with its less sophisticated counterpart.

This outcome reflects the fact that TAD backbones and detectors are not developed in a shared design space. I3D features became a de facto standard in the TAD community, and detection heads, including ActionFormer, were iteratively optimized against these features over multiple publication cycles. Hyperparameters such as regression ranges, pyramid

scales, learning rate schedules, and loss weighting were tuned, either explicitly or implicitly through architecture search, to match the statistical properties of I3D outputs. VideoMAE v2 was developed in the self-supervised video representation community with different evaluation paradigms (primarily action classification on trimmed clips), and its features were never systematically validated as inputs to anchor-free temporal detectors.

The result is that “better backbone” and “better detector” are not context-free designations. Each component is better *within the ecosystem in which it was developed*. Transplanting a component across ecosystems without recalibration breaks the implicit assumptions that made it effective.

C. The Baseline as a Diagnostic Tool

The CNN baseline plays a useful diagnostic role in this study. Because it is a shallow, generic architecture with no feature-specific tuning, its relative performance across backbones may reflect the intrinsic quality of the features more directly than ActionFormer’s does. The baseline performs $2.4\times$ better with VideoMAE v2 than with I3D (4.04% vs. 1.66%), which could indicate that VideoMAE v2 features contain stronger action-background discriminative signals. However, as noted in Section IV, the stride difference between these two conditions (8 vs. 16) prevents a clean attribution: the higher temporal resolution alone could account for part or all of the observed gap.

Regardless of the stride confound, this comparison points toward a practical recommendation: when evaluating a new feature backbone for TAD, performance should be assessed on a simple, untuned detector before drawing conclusions about feature quality. Strong performance on a tuned detector may reflect feature-detector compatibility rather than intrinsic feature informativeness.

D. Limitations

This study has several limitations that constrain the generalizability of its conclusions.

The most significant limitation is the presence of confounding variables across experimental conditions. Feature stride, pyramid depth, and detection head capacity all vary between the ActionFormer and baseline configurations (see Table I). These differences were preserved intentionally to evaluate each detector in its designed operating regime, but they prevent fully causal attribution of performance differences to any single factor. In particular, the stride variation across backbone conditions (stride 4 vs. 8 for ActionFormer, stride 8 vs. 16 for the baseline) means that backbone comparisons are confounded with temporal resolution differences. A cleaner comparison would require running all four conditions at matched strides and pyramid depths, which we leave to future work.

Additionally, no hyperparameter adaptation was performed for the VideoMAE v2 conditions. All training settings, including learning rate, regression ranges, loss normalization, and architectural parameters, were inherited from configurations

optimized for I3D. It is likely that targeted tuning for VideoMAE v2 features would substantially improve performance, though the magnitude of improvement that can be recovered through tuning alone remains an open question.

Finally, this study is limited to a single dataset (THUMOS14) and two backbones. The observed coupling patterns may differ on datasets with different action densities, video lengths, or class distributions, and with other backbone architectures such as SlowFast [10] or X3D [11].

VI. CONCLUSION AND FUTURE WORK

This work presented a controlled 2×2 study comparing two feature backbones (I3D and VideoMAE v2) and two temporal detection heads (a CNN baseline and ActionFormer) on the THUMOS14 temporal action detection benchmark. By holding the detection framework, training objective, and evaluation protocol constant across conditions, we isolated the effect of each component on detection performance, though confounding variables including feature stride and pyramid depth limit the strength of causal claims.

Our results demonstrate that ActionFormer achieves strong performance (66.76% average mAP) only when paired with I3D features and collapses to near-baseline levels (4.57%) with VideoMAE v2 features. Meanwhile, the simpler CNN baseline suggests that VideoMAE v2 may produce stronger general-purpose features than I3D (4.04% vs. 1.66%), though this comparison is confounded by a stride difference that gives VideoMAE v2 twice the temporal resolution. These findings indicate that gains in video representation learning and temporal detection modeling are not additive by default, and that effective TAD systems likely require co-design of the backbone and detector rather than independent component substitution.

A. Future Work

Several directions follow naturally from the limitations identified in this study.

Controlled stride and pyramid experiments. The most direct extension would re-run all four conditions at a matched feature stride (e.g. stride 8 for all backbones) and a matched number of pyramid levels. This would eliminate the primary confounding variables and enable stronger causal claims about backbone–detector interaction.

Feature adaptation layers. Rather than relying on a single linear projection to bridge backbone feature spaces, future work could investigate lightweight adaptation modules, such as a learned multi-layer projection, feature normalization layers calibrated to each backbone’s output distribution, or a small fine-tuning stage on the embedding network, to align VideoMAE v2 features with the detector’s expected input statistics.

Hyperparameter retuning for VideoMAE v2. A systematic hyperparameter search over regression ranges, learning rate, and loss normalization specifically for VideoMAE v2 features would clarify how much of the observed performance collapse is due to fundamental feature incompatibility versus suboptimal training configuration.

Additional backbones and datasets. Extending the factorial design to include additional backbones (e.g. SlowFast [10], X3D [11], InternVideo) and datasets beyond THUMOS14 (e.g. ActivityNet, HACS) would test whether the coupling phenomenon generalizes or is specific to the I3D–ActionFormer pairing on sports videos.

Feature distribution analysis. A quantitative comparison of the statistical properties of I3D and VideoMAE v2 features, including activation distributions, temporal autocorrelation, and class separability in the embedding space, would help explain *why* certain features are compatible with certain detectors at a mechanistic level, informing the design of more backbone-agnostic detection heads.

REFERENCES

- [1] H. Idrees, A. R. Zamir, Y.-G. Jiang, A. Gorban, I. Laptev, R. Sukthankar, and M. Shah, “The THUMOS challenge on action recognition for videos ‘in the wild’,” *Comput. Vis. Image Underst.*, vol. 155, pp. 1–23, 2017.
- [2] V. Escorcia, F. C. Heilbron, J. C. Niebles, and B. Ghanem, “DAPs: Deep action proposals for action understanding,” in *Proc. ECCV*, ser. LNCS, vol. 9907, Springer, 2016, pp. 768–784.
- [3] T. Lin, X. Zhao, H. Su, C. Wang, and M. Yang, “BSN: Boundary sensitive network for temporal action proposal generation,” in *Proc. ECCV*, ser. LNCS, vol. 11208, Springer, 2018, pp. 3–19.
- [4] T. Lin, X. Liu, X. Li, E. Ding, and S. Wen, “BMN: Boundary-matching network for temporal action proposal generation,” in *Proc. IEEE/CVF ICCV*, 2019, pp. 3889–3898.
- [5] C.-L. Zhang, J. Wu, and Y. Li, “ActionFormer: Localizing moments of actions with transformers,” in *Proc. ECCV*, ser. LNCS, vol. 13664, Springer, 2022, pp. 492–510.
- [6] T. N. Tang, K. Kim, and K. Sohn, “TemporalMaxer: Maximize temporal context with only max pooling for temporal action localization,” *arXiv preprint arXiv:2303.09055*, 2023.
- [7] K. Simonyan and A. Zisserman, “Two-stream convolutional networks for action recognition in videos,” in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 27, 2014, pp. 568–576.
- [8] D. Tran, L. Bourdev, R. Fergus, L. Torresani, and M. Paluri, “Learning spatiotemporal features with 3D convolutional networks,” in *Proc. IEEE ICCV*, 2015, pp. 4489–4497.
- [9] J. Carreira and A. Zisserman, “Quo vadis, action recognition? A new model and the Kinetics dataset,” in *Proc. IEEE CVPR*, 2017, pp. 6299–6308.
- [10] C. Feichtenhofer, H. Fan, J. Malik, and K. He, “SlowFast networks for video recognition,” in *Proc. IEEE/CVF ICCV*, 2019, pp. 6202–6211.
- [11] C. Feichtenhofer, “X3D: Expanding architectures for efficient video recognition,” in *Proc. IEEE/CVF CVPR*, 2020, pp. 203–213.
- [12] K. He, X. Chen, S. Xie, Y. Li, P. Dollár, and R. Girshick, “Masked autoencoders are scalable vision learners,” in *Proc. IEEE/CVF CVPR*, 2022, pp. 15979–15988.
- [13] Z. Tong, Y. Song, J. Wang, and L. Wang, “VideoMAE: Masked autoencoders are data-efficient learners for self-supervised video pre-training,” in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 35, 2022, pp. 10078–10093.
- [14] L. Wang, B. Huang, Z. Zhao, Z. Tong, Y. He, Y. Wang, Y. Wang, and Y. Qiao, “VideoMAE V2: Scaling video masked autoencoders with dual masking,” in *Proc. IEEE/CVF CVPR*, 2023, pp. 14549–14560.
- [15] Y. Abu Farha and J. Gall, “MS-TCN: Multi-stage temporal convolutional network for action segmentation,” in *Proc. IEEE/CVF CVPR*, 2019, pp. 3575–3584.
- [16] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, “Focal loss for dense object detection,” in *Proc. IEEE/CVF ICCV*, 2017, pp. 2980–2988.
- [17] S. Liu, C.-L. Zhang, C. Zhao, and B. Ghanem, “OpenTAD: A unified framework and comprehensive study of temporal action detection,” in *Proc. IEEE/CVF CVPRW*, 2025, pp. 2625–2635.
- [18] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2019.
- [19] D. Shi, Y. Zhong, Q. Cao, L. Ma, J. Li, and D. Tao, “TriDet: Temporal action detection with relative boundary modeling,” in *Proc. IEEE/CVF CVPR*, 2023, pp. 18857–18866.